VUE 3 COMPOSITION API

Reactivity Fundamentals

Veri değişikliklerini otomatik olarak arayüze yansıtan JavaScript Proxy tabanlı sistem.

Reaktiflik Nedir?

State değiştiğinde DOM'un otomatik olarak güncellenmesi mekanizması.



JavaScript Proxy Temelleri

Vue 3, nesne erişimlerini yakalamak için yerleşik Proxy API'sini kullanır.

Get (Okuma) Operasyonu

Değer okunduğunda Vue hangi bileşenin bu veriye bağımlı olduğunu kayda geçer.

`Dependency Tracking`

Set (Yazma) Operasyonu

Değer değiştirildiğinde Vue ilgili bağımlılıkları tetikleyerek render sürecini çalıştırır.

`Trigger Effects`

Primitif Değerler İçin: ref()

Her tür veriyi sarmalayan reaktif kutular oluşturur.

JavaScript Kullanımı

```
import { ref } from 'vue'

 const count = ref(0)
 count.value++ // .value şart
```

Template Kullanımı

```
    {{ count }}{{ count }}

 // .value otomatik açılır
```

Nesneler İçin: reactive()

Kompleks nesne ve dizileri doğrudan reaktif Proxy nesnesine dönüştürür.

Doğrudan Erişim

```
import { reactive } from 'vue'

 const state = reactive({ count: 0 })
 state.count++ // .value gerekmez
```

Önemli Kurallar

Yalnızca nesne ve dizilerle çalışır.

Tüm nesne baştan atamayla reaktiflik kopar.

ref() vs reactive()

İki reaktif state tanımlama yönteminin temel farkları.

Özellik	ref()	reactive()
Veri Tipleri	Primitif ve Nesne	Sadece Nesne / Dizi
Erişim Şekli	.value gerekli	Doğrudan özellik erişimi
Yeniden Atama	Desteklenir	Desteklenmez

Derin Reaktiflik (Deep Reactivity)

İç içe geçmiş nesne ve diziler otomatik olarak reaktif hale getirilir.

İç İçe Nesne Yapısı

```
const user = reactive({
   profile: {
     name: 'Ahmet'
   }
 })
 user.profile.name = 'Mehmet' // Tetikler
```

Avantajı

Derin seviyedeki mülk değişiklikleri bile UI bileşenlerini anında günceller.

`Automatic Deep Proxy`

Sık Yapılan Hatalar ve Tuzaklar

Reaktiflik bağının kopmasına yol açabilecek durumlar.

Destructuring (Parçalama)

reactive nesneyi destructure etmek bağı koparır. toRefs() kullanılmalıdır.

Doğrudan Atama

reactive değişkenlere yeni bir nesne referansı atamak reaktifliği yok eder.

Mutasyon Eksikliği

Dizi metodlarında doğrudan indeks atamaları yerine uygun metodlar seçilmelidir.

En İyi Uygulamalar (Best Practices)

Temiz ve güvenilir Vue 3 uygulamaları geliştirme rehberi.

Varsayılan Olarak ref()

Tür bağımsızlığı ve yeniden atama esnekliği nedeniyle öncelikli olarak ref() tercih edin.

Form ve Gruplar İçin reactive()

Birbiriyle ilişkili form alanları ve veri modelleri için reactive() kullanın.

Teşekkürler

Sorularınız ve katkılarınız için teşekkür ederim.